CS541: Applied Machine Learning, Spring 2026, Lab 1

Lab 1 is an exercise that introduces the Numpy Python library. When implementing machine learning models, you will often find yourself having to perform complicated operations on matrices or arrays of data and/or model weights. Numpy implements many high-level mathematical functions that operate on these arrays and matrices.

The goal of this is to gain a basic understanding of how Numpy represents matrices and some operations upon them. Many machine learning libraries are also designed to operate like Numpy or are directly implemented using Numpy, making it an essential tool to understand for any practitioner.

Lab Grading

Labs are hands-on exercises designed to provide guided experience in key concepts through this class. You are graded based on in-lab participation (not correctness), and **are required to submit** your lab work after class, before Friday of that week. *Make sure you fill out the attendence form before leaving class*.

For students who miss a lab, you can submit a make-up lab on gradescope by the Friday directly following the lab for partial credit. Please see the syllabus for the lab grading policy.

Part 1: Basic Numpy Operations

We will begin by learning how to perform some simple operations upon matrices. Let's say we are given a dataset with three features and five observations. Each observation can be thought of as a sample in the dataset, whereas the features are variables representing some captured information. For example, an observation could be an image of a cat, whereas a variable could represent measured values such as the likelihood you see its ears or tail in the photo. Thus, our dataset would have the following form:

$$\begin{bmatrix} 0.1 & 0.3 & 0.8 \\ 0.4 & 0.7 & 0.6 \\ 0.9 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.7 \\ 0.2 & 0.9 & 0.5 \end{bmatrix}$$

This would be implemented using the array function in Numpy.

```
import numpy as np

arr = np.array([[0.1, 0.3, 0.8], [0.4, 0.7, 0.6], [0.9, 0.2, 0.3], [0.4, 0.5, 0.7], [0.2, 0.9, 0.5]])
print(arr)
```

```
[[0.1 0.3 0.8]
 [0.4 0.7 0.6]
 [0.9 0.2 0.3]
 [0.4 0.5 0.7]
 [0.2 0.9 0.5]]
```

Slicing an Array:

Let's say we decided we wanted to remove the first two samples of the dataset because we found its features were not reliability measured. For this purpose, we will use array indexing functions. If we index into the array but leave a ":" after the index number (i.e., "1:"), then it assumes that you want to keep everything afterwards. If you want to slice off the last item, then you put the colon before the index number. Let's try it out!

```
# write code to remove the first two observations in the matrix
sliced_arr = arr[2:]
print(sliced_arr)
```

```
[[0.9 0.2 0.3]
 [0.4 0.5 0.7]
 [0.2 0.9 0.5]]
```

Notice that we created a new Python variable to represent the sliced matrix. Thus, our original matrix still contains the entire dataset:

```
print(arr.shape)
arr
```

```
(5, 3)
array([[0.1, 0.3, 0.8],
       [0.4, 0.7, 0.6],
       [0.9, 0.2, 0.3],
       [0.4, 0.5, 0.7],
       [0.2, 0.9, 0.5]])
```

Now lets try to remove the first two observations and the last feature:

```
# write code to remove the first two observations and the last
# feature column of the matrix

sliced_arr = arr[2:, :-1]
print(sliced_arr)
```

```
[[0.9 0.2]
 [0.4 0.5]
 [0.2 0.9]]
```

Searching and Counting:

Sometimes we'd like to find features or predictions that represent a particular value. For this purpose you can use logical functions and functions like "sum" or "count_nonzero" to implement this.

```
arr = np.array([1, 2, 2, 3, 3, 4, 5, 5])

# write code to count the number of times "3" appears in the
# array above

count_of_3 = np.sum(arr == 3)
print(count_of_3)
```

```
2
```

Stacking Arrays:

After making predictions on two observations, you may want to stack the vector of predictions to make a single matrix. Let's practice that next using the np.stack function!

```
prediction_vector1 = np.array([1, 2, 3])
prediction_vector2 = np.array([4, 5, 6])

# write code that stacks prediction_vector1 on top of
# prediction_vector2
stacked_arr = np.stack((prediction_vector1, prediction_vector2), axis=0)
print(stacked_arr)
```

```
[[1 2 3]
 [4 5 6]]
```

Applying Element-wise Functions:

Many functions are implemented in numpy to perform elementwise operations. Let's try using np.sqrt to take the square root of each element of a vector.

```
arr = np.array([1, 2, 3, 4])

# write code to perform an elementwise sqrt on "arr"
square_root = np.sqrt(arr)
print(square_root)
```

```
[1.         1.41421356 1.73205081 2.        ]
```

Matrix/Vector Arithmetic

Most of the time you should use built-in functions for performing matrix operations. Experienced practitioners will attempt to vectorize their code, i.e., replacing loops with matrix functions. However, if you are not experienced with the functions, it can be confusing. Let's try to perform a dot product of two vectors. Since this is written as multiplication, let's begin by trying to use "*" to represent multiplication.

```
vec1 = np.array([1, 2, 3])
vec2 = np.array([4, 5, 6])

# write code that multiples vec1 by vec2
dot_product_attempt1 = vec1 * vec2
print(dot_product_attempt1)
```

```
[ 4 10 18]
```

That didn't quite work out! What happened is that in Numpy a "*" refers to an elementwise product, i.e., each matrix must be the same size (or it'll throw an error) and then the corresponding entries are multiplied together. To perform matrix operations, we need to use a function. Let's try this again using np.dot.

```
# write code that uses np.dot to perform a dot
# product on vec1 and vec2
dot_product_attempt2 = np.dot(vec1, vec2)
print(dot_product_attempt2)
```

```
32
```

Cosine similarity

Now that we've gone through some basics, let's try to implement a simple formula. Cosine similarity is often used to measure the distance between two vectors. Implement the operation using only numpy functions/without a loop. For reference, here is the formula:

$$CosSim = \frac{A \cdot B}{\|A\|\|B\|}$$

Some helpful functions

```
vec1 = np.array([1, 2, 3])
vec2 = np.array([4, 5, 6])

# write code to compute the cosine similarity of vec1
# and vec2

dot_product = np.dot(vec1, vec2)
norm1 = np.linalg.norm(vec1)
norm2 = np.linalg.norm(vec2)

cosine_similarity = dot_product / (norm1 * norm2)

print(cosine_similarity)
```

```
0.9746318461970762
```

Part 2: Dataset setup

Before you can implement a machine learning model, we first need to get a dataset within the proper format. In this section, we'll practice some of the basic operations for accomplishing this.

As a first step, we need to load the dataset. In this case let's use a standard dataset, Iris flower, which is composed of attributes of 3 different types of irises'. As before, the rows are the observations and the columns are composed of different attributes like Sepal length, Petal Length and so on.

```
from sklearn.datasets import load_iris
iris = load_iris()

# the matrix containing the features of the dataset
iris_X = iris.data
print('dataset', type(iris_X))

# the target labels of the dataset
iris_y = iris.target
print('labels', type(iris_y))
```

```
dataset <class 'numpy.ndarray'>
labels <class 'numpy.ndarray'>
```

Taking a closer look

Upon receiving a new dataset, one of the first things you'll want to do is to get a better understanding of what it contains. We see above that this dataset was loaded into numpy arrays. Now, as a first step, write some code to identify how many observations there are in the dataset and what the size of the corresponding label set is as well.

```
# write code to print the size of the dataset and labels
print("dataset size (rows, cols):", iris_X.shape)
print("labels size:", iris_y.shape)
```

```
dataset size (rows, cols): (150, 4)
labels size: (150,)
```

Now that we see the size of the dataset, we'd like to get a better understanding of what each of the values means. Luckily this is easily accessable in our 'iris' variable:

```
print("Feature names:", iris.feature_names)
print("Target names:", iris.target_names)
```

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target names: ['setosa' 'versicolor' 'virginica']
```

Inspecting the data

Now that we have an idea of what the size is and what each of the values means, let's take a look at the data itself. Let's write some code that let's us inspect the first five observations in both the dataset and labels.

```
# write code to print the first five observations of the dataset
# and labels variables
print("First 5 observations (iris_X):")
print(iris_X[:5])

print("First 5 labels (iris_y):")
print(iris_y[:5])
```

```
First 5 observations (iris_X):
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
First 5 labels (iris_y):
[0 0 0 0 0]
```

As we can see the first five labels are all the same. This suggests that perhaps the dataset has grouped together common classes. However, just to be sure, let's check how many labels there are:

```
print(np.unique(iris_y))
```

```
[0 1 2]
```

Luckily we see that each category is present in the dataset. This still might mean the data is imbalanced, where some categories are more common, but we will discuss such cases later in the semester.

Split the dataset into train/test/validation

Since the dataset looks good so far, the next step we'll want to take is to split the dataset into three parts: training data, testing data, and validation data. As we'll discuss in class, this is a critical part of setting up a machine learning experiment. We use only training and validation data to select the right model to use and to set all hyperparameters. The testing data is only used to verify that the model created using our training and validation data works on unseen samples. Not following this protocol can be costly as models that are

tuned using the test set may not work in practice. This is often referred to as train-test contamination, and any good ML practitioner must be vigilant in avoiding it.

A common way of splitting the data is using 70% training, 20% testing, and 10% validation, but many other splits may be used depending on the dataset. Let's implement a function to split our dataset according to these specifications

```
def get_train_val_test_split(iris_X,iris_y, seed=42):
    # write code to split the dataset into 70/10/20 train/val/test
    # Since the dataset labels are in order, you should first randomly
    # shuffle the dataset observations, but remember that iris_x and iris_y
    # need to be split in the same way so you can't use np.random.shuffle
    # directly on iris_X or iris_y
    n = iris_X.shape[0]

    # make a single shuffled index array
    rng = np.random.default_rng(seed)
    idx = rng.permutation(n)

    X_shuffled = iris_X[idx]
    y_shuffled = iris_y[idx]

    # sizes: 70/10/20
    train_end = int(0.7 * n)
    val_end = train_end + int(0.1 * n)  # next 10%

    train_X, train_y = X_shuffled[:train_end], y_shuffled[:train_end]
    val_X, val_y     = X_shuffled[train_end:val_end], y_shuffled[train_end:val_end]
    test_X, test_y   = X_shuffled[val_end:], y_shuffled[val_end:]


    return (train_X,train_y),(val_X,val_y),(test_X,test_y)

train_split,val_split,test_split = get_train_val_test_split(iris_X,iris_y)
```

```
X_train_split,y_train_split = train_split
X_val_split,y_val_split = val_split
X_test_split,y_test_split = test_split
```

Let's look at the shapes of our data to ensure it's correct

```
def get_split_shapes(data_split,tag):
  print('X' + tag)
  print(data_split[0].shape)
  print('y' + tag)
  print(data_split[1].shape)
```

```
get_split_shapes(train_split,'_train')
```

```
X_train
(105, 4)
y_train
(105,)
```

```
get_split_shapes(val_split,'_val')
```

```
X_val
(15, 4)
y_val
(15,)
```

```
get_split_shapes(test_split,'_test')
```

```
X_test
(30, 4)
y_test
(30,)
```

Standardize the dataset

The final step in setting up a dataset is to standardize it such that each of the features have the same scale (we will discuss why this is important later in the semester). We will go over many ways of doing this, but one common way is making the data zero-mean and unit variance. In other words, you want each feature to be subtracted from its mean value across the training set and divided by its standard deviation (some helpful functions).

```
# Write some code to make each feature to have zero mean and unit variance
# across the dataset.  To avoid train-test contamination, compute the mean and
# standard deviation using only the training data, but apply those computed
# values to the test and validation data as well.

train_iris_X_normalized = (X_train_split - np.mean(X_train_split, axis=0)) / np.std(X_train_split, axis=0)
val_iris_X_normalized = (X_val_split - np.mean(X_train_split, axis=0)) / np.std(X_train_split, axis=0)
test_iris_X_normalized = (X_test_split - np.mean(X_train_split, axis=0)) / np.std(X_train_split, axis=0)
```

Finally, let's look we look at the normalized data to ensure it looks correct

```
print(train_iris_X_normalized[:5])
print(y_train_split[:5])
```

```
[[ 0.01818635  0.47783003  0.52492467  0.78863921]
 [-0.22050951 -1.0348982  -0.23329985 -0.31361859]
 [ 0.25688221 -1.0348982   0.99152437  0.23751031]
 [-1.53333674  0.22570866 -1.39979912 -1.41587639]
 [ 0.49557807 -0.78277683  0.58324963  0.78863921]]
[1 1 2 0 2]
```

```
print(val_iris_X_normalized[:5])
print(y_train_split[:5])
```

```
[[-0.81724916  2.74692236 -1.39979912 -1.55365861]
 [ 0.01818635 -0.02641271  0.17497489  0.37529254]
 [-1.17529295 -1.53914094 -0.34994978 -0.31361859]
 [ 0.614926    0.47783003  0.81654949  1.47755034]
 [-0.93659709  0.7299514  -1.28314919 -1.00252971]]
[1 1 2 0 2]
```

```
print(test_iris_X_normalized[:5])
print(y_train_split[:5])
```

```
[[ 2.16644909  1.99055825  1.63309897  1.33976811]
 [-0.5785533   2.24267962 -1.51644904 -1.14031194]
 [-0.93659709  1.99055825 -1.16649926 -1.14031194]
 [ 1.09231772 -0.02641271  0.93319941  1.20198589]
 [ 0.13753428 -0.02641271  0.52492467  0.78863921]]
[1 1 2 0 2]
```